SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

DocuMind - Research Assistant Agent

Lab CA-1
AI

Shreya Menon 16010123324

Shreyans Tatiya 16010123325

Siddhant Raut 16010123331

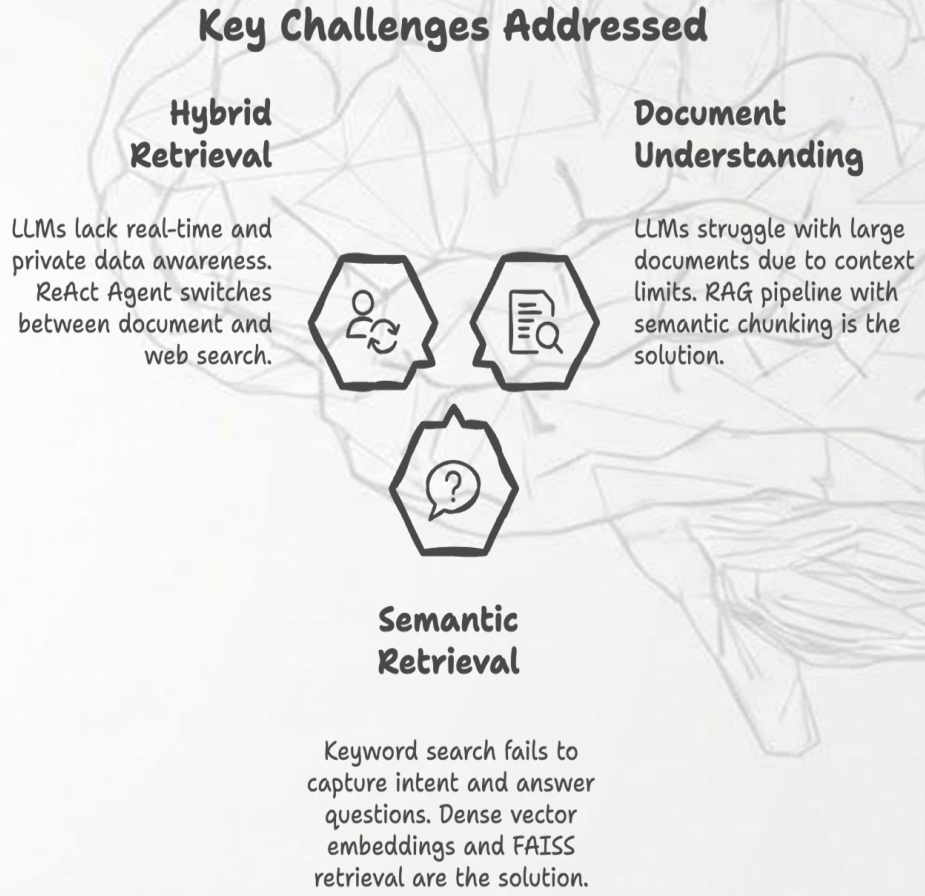
Problem & Motivation

The Problem: Information Overload

- Users deal with large, dense documents (PDFs, research papers, reports)
- Traditional search (Ctrl+F) fails for complex, context-based queries
- Difficult to both **extract insights and answer specific questions**

Outcome

- Enables **natural language Q&A over documents**
- Faster, smarter, and more reliable knowledge extraction
- Combines **local + external intelligence** seamlessly



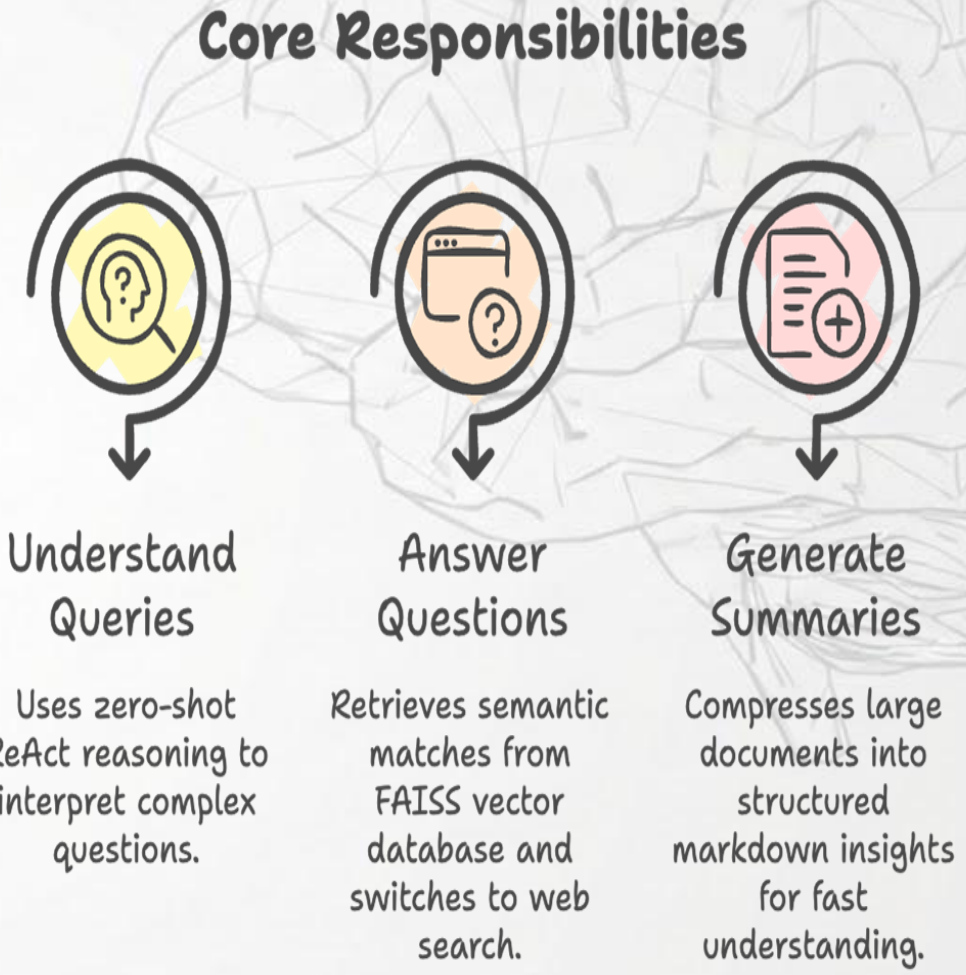
Agent Role

Goal of the Agent

- Bridge the gap between static data (documents) and dynamic user queries
- Transform natural language queries into intelligent search operations
- Return structured insights instead of raw text

Agent Role: AI Research Assistant

- Acts as an autonomous intermediary between user and data
- Dynamically orchestrates tools:
  - document_search
  - summarize_document
  - web_search



Agent Design - Core Concepts

1. Agent Type

🦜🔗 LangChain: ZERO_SHOT_REACT_DESCRIPTION

- No hardcoded logic or fine-tuning
- Selects tools dynamically using tool descriptions + query intent

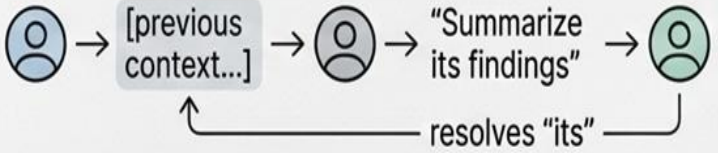
2. Planning Strategy — ReAct Loop

Follows multi-step reasoning cycle:

Repeats until final structured response is generated

3. Memory (Short-Term Context)

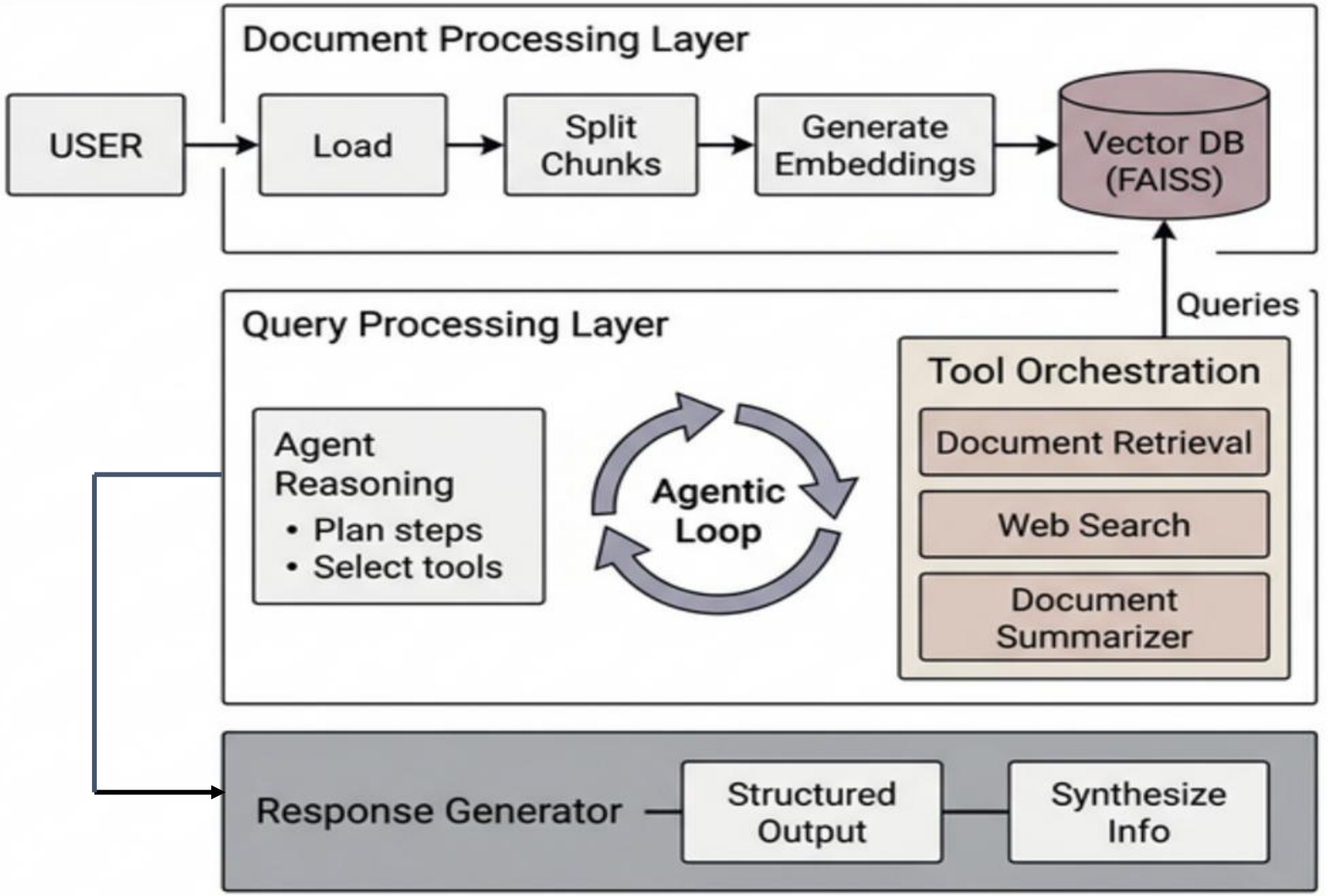
- Maintains conversation history buffer
- Enables contextual understanding of follow-up queries



4. Tools Orchestrated by Agent

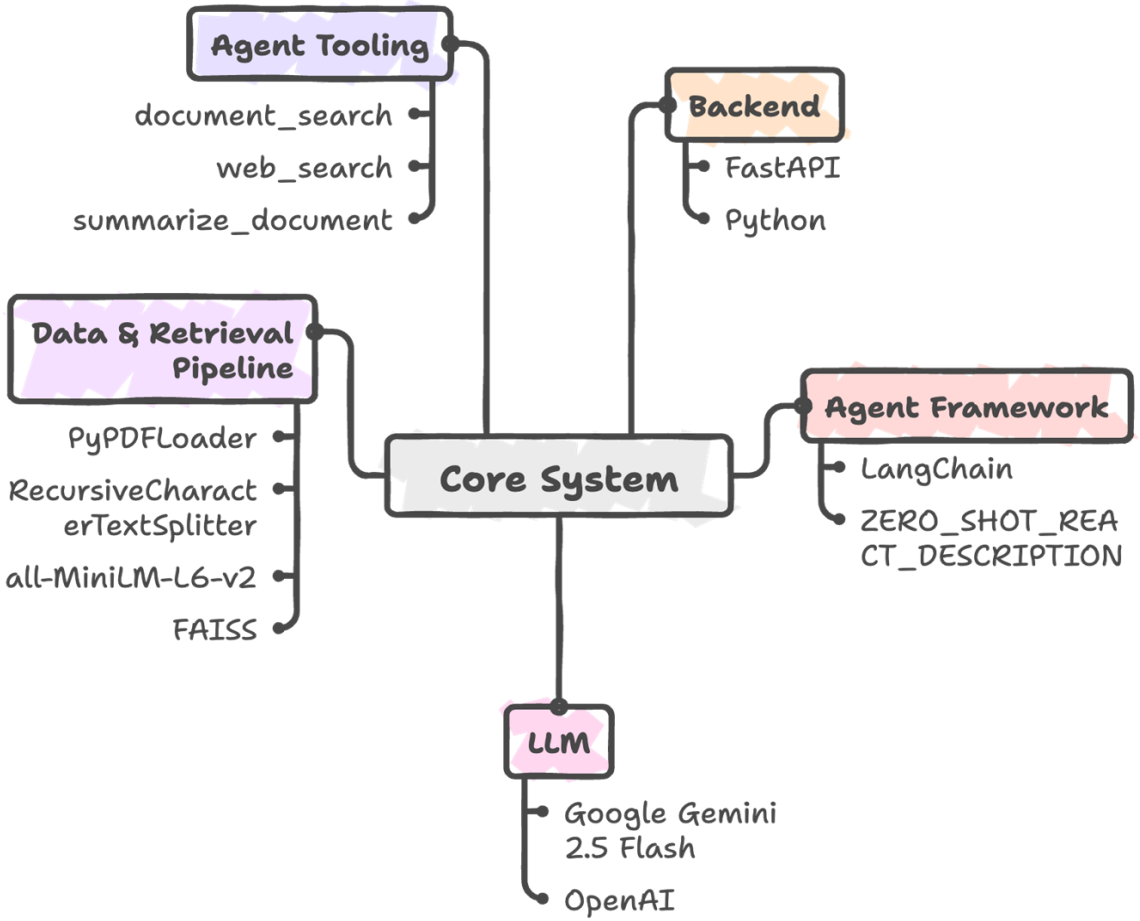
`document_search`	→ FAISS + MiniLM for semantic retrieval from PDFs
`web_search`	→ DuckDuckGo for real-time external data
`summarize_document`	→ Full-document compression for high-level insights

Architecture Diagram

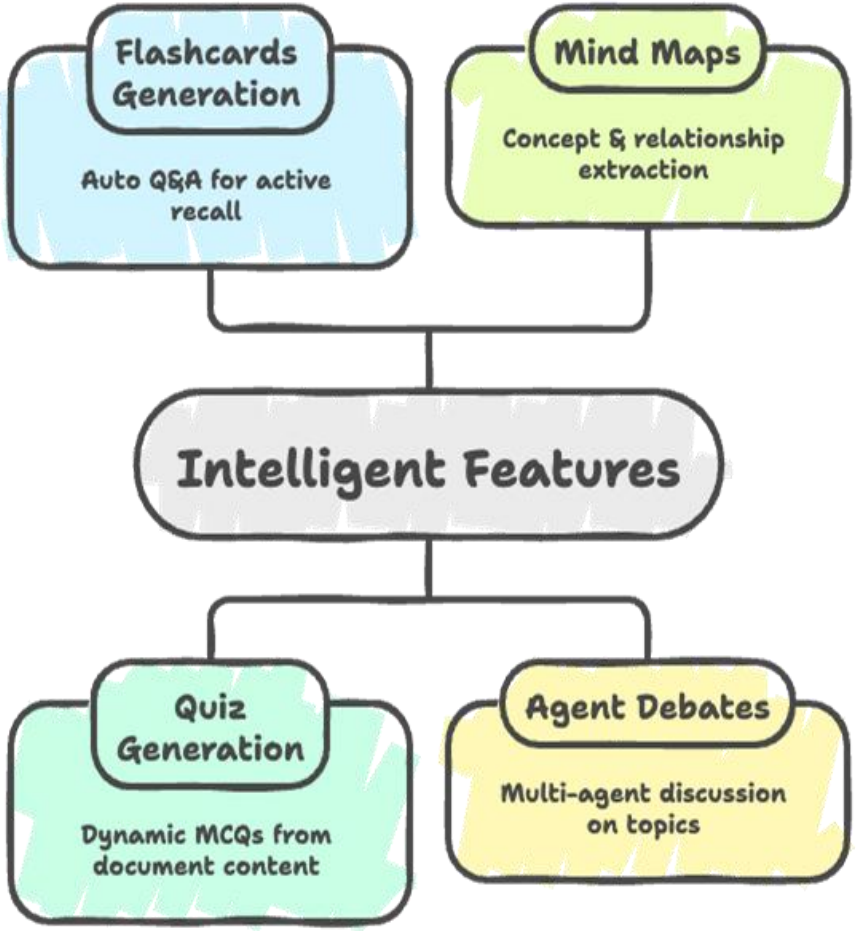


Implementation

Core System Architecture and Components



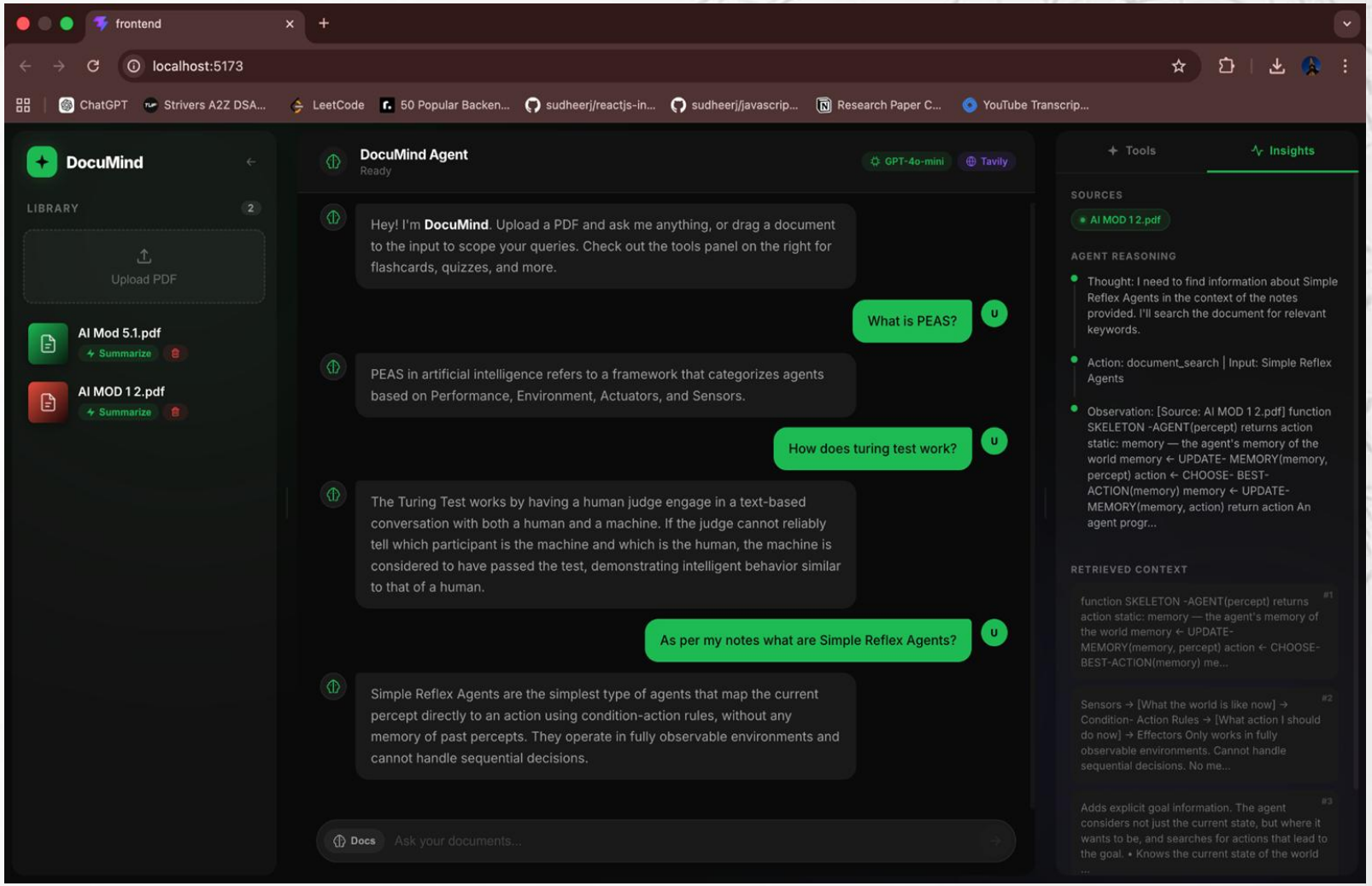
Enhancing Learning with Intelligent Features



Execution with Trace / Reasoning

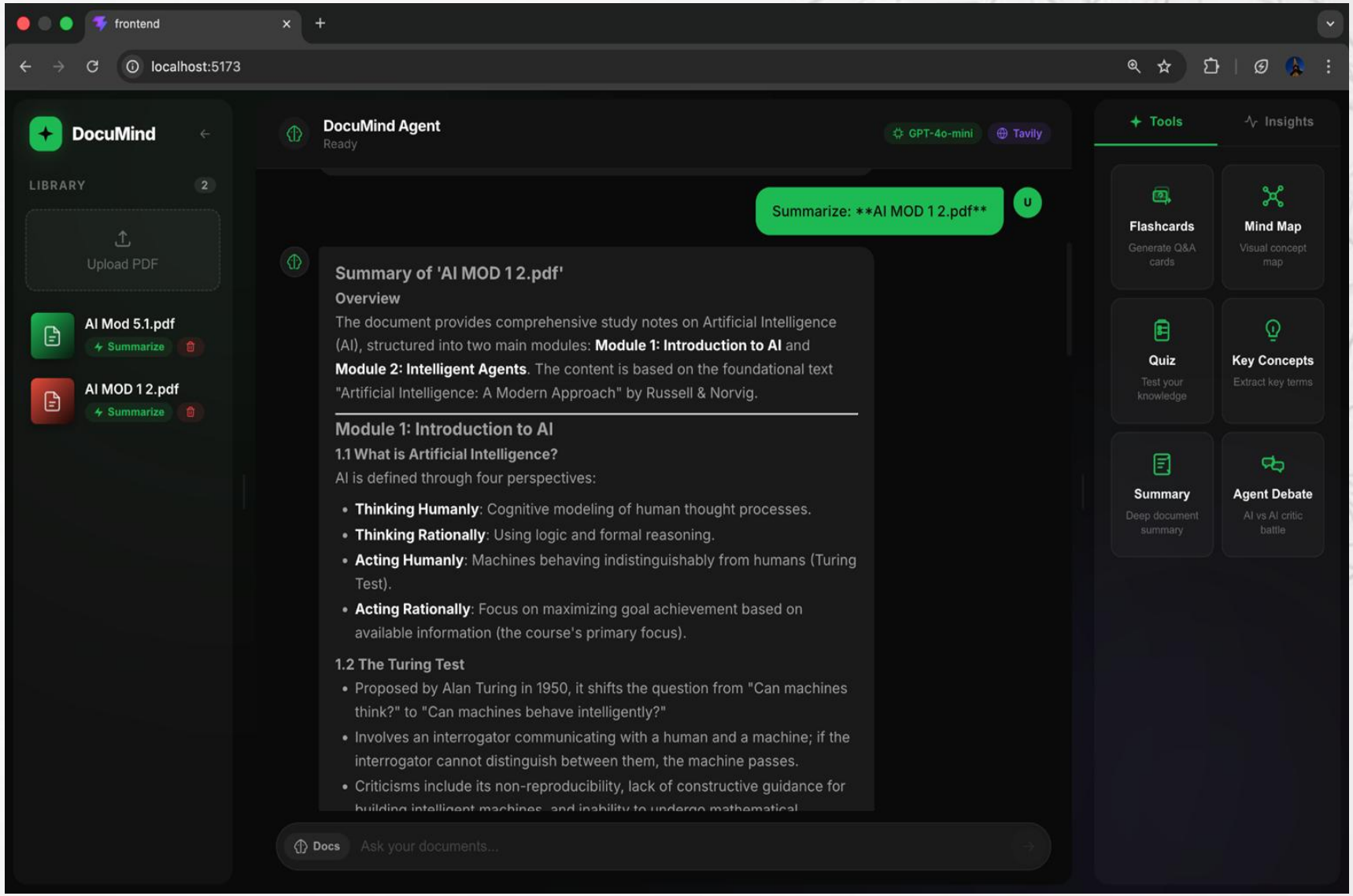
Document Search

- The agent follows a **ReAct reasoning loop** (Thought → Action → Observation)
- It first identifies the need for **document-based information**
- Selects the **document_search tool** to retrieve relevant content
- Observes retrieved data and generates a **context-aware answer**
- Ensures transparency through **visible reasoning traces and source citations**



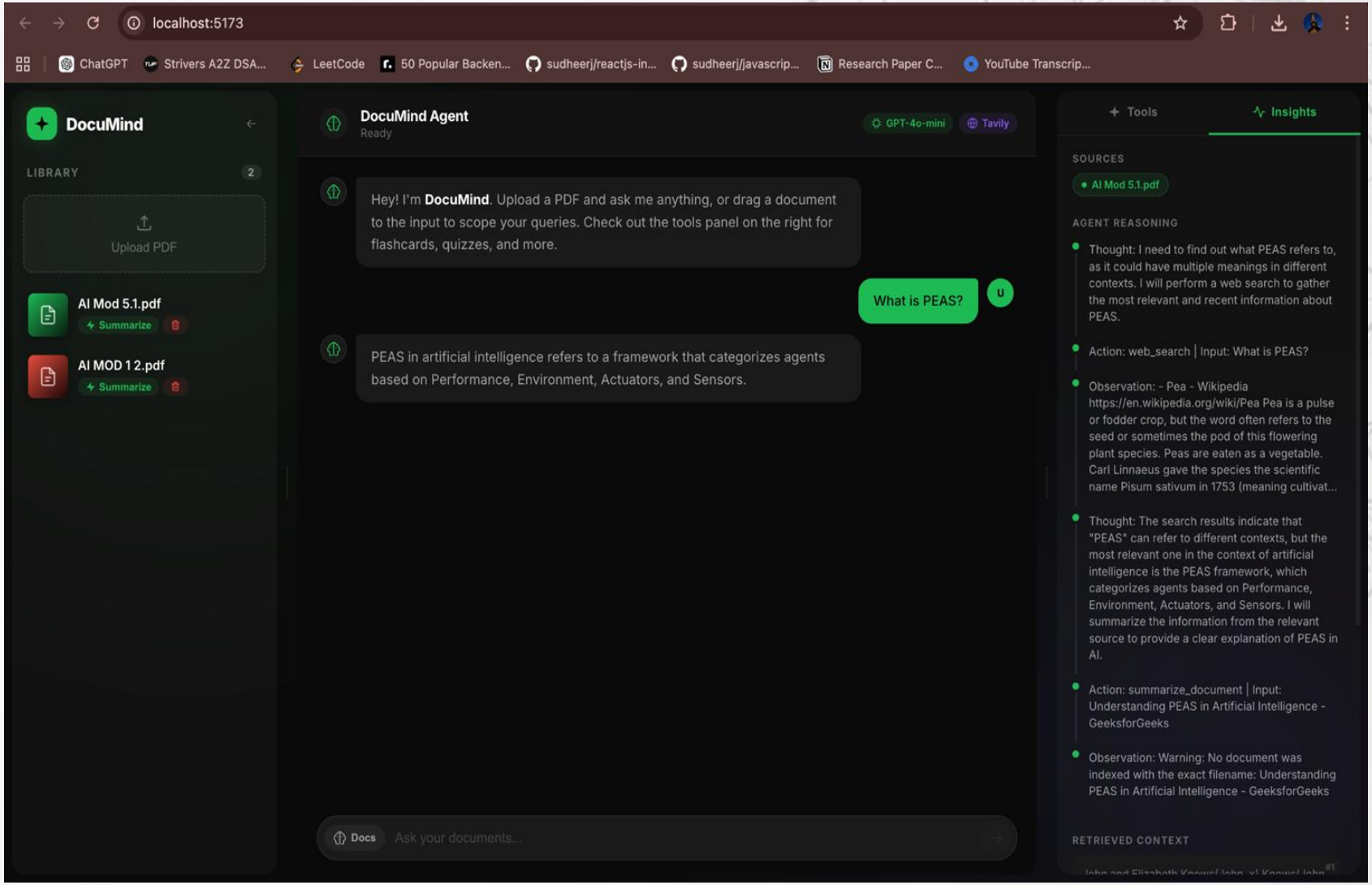
Summarize

- Generates structured summary from document
- Uses RAG + LLM reasoning
- Triggered via summarize_document tool
- Outputs clear, concise insights



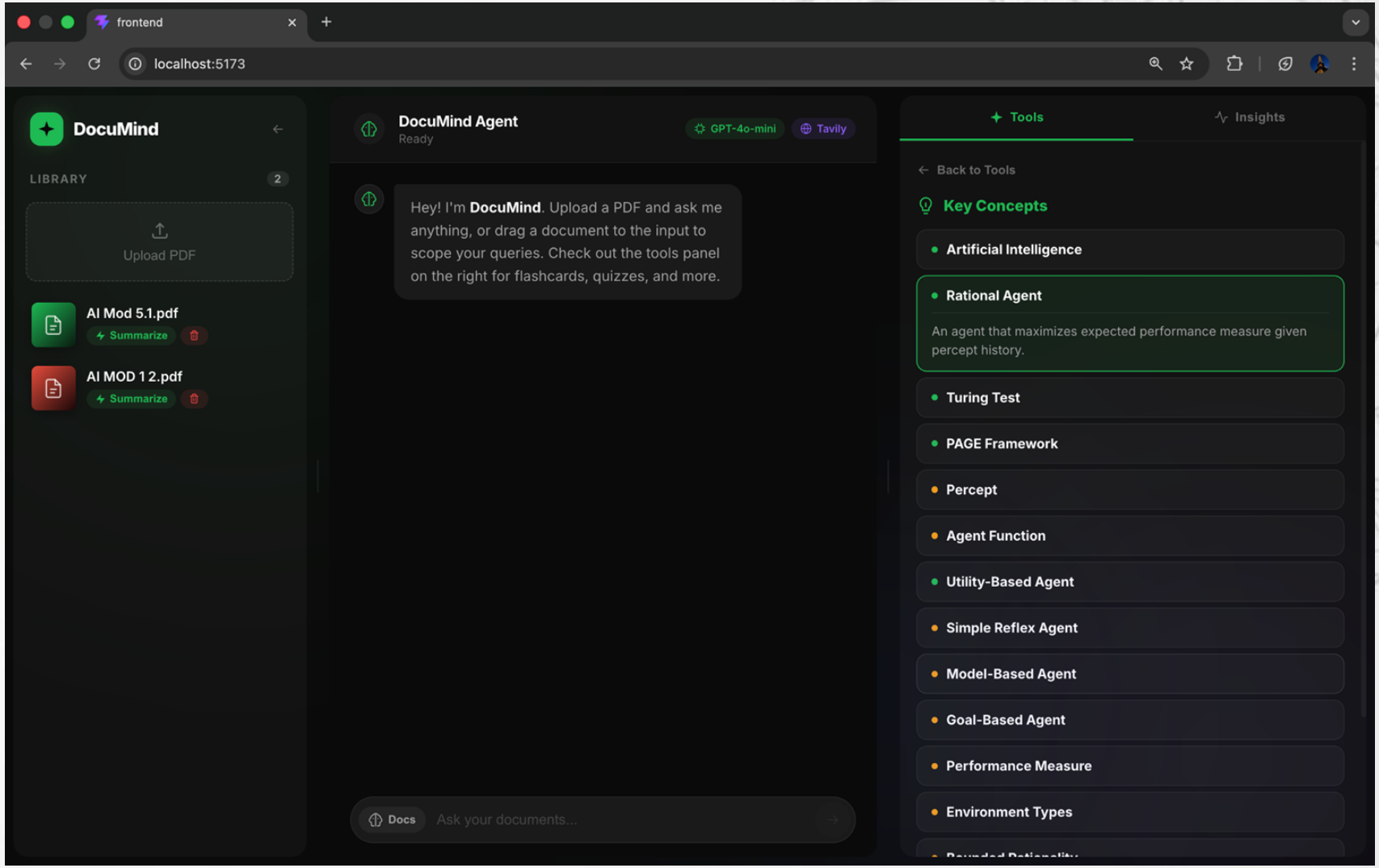
Web Search + Reasoning

- Agent detects missing knowledge
- Uses web_search via ReAct reasoning



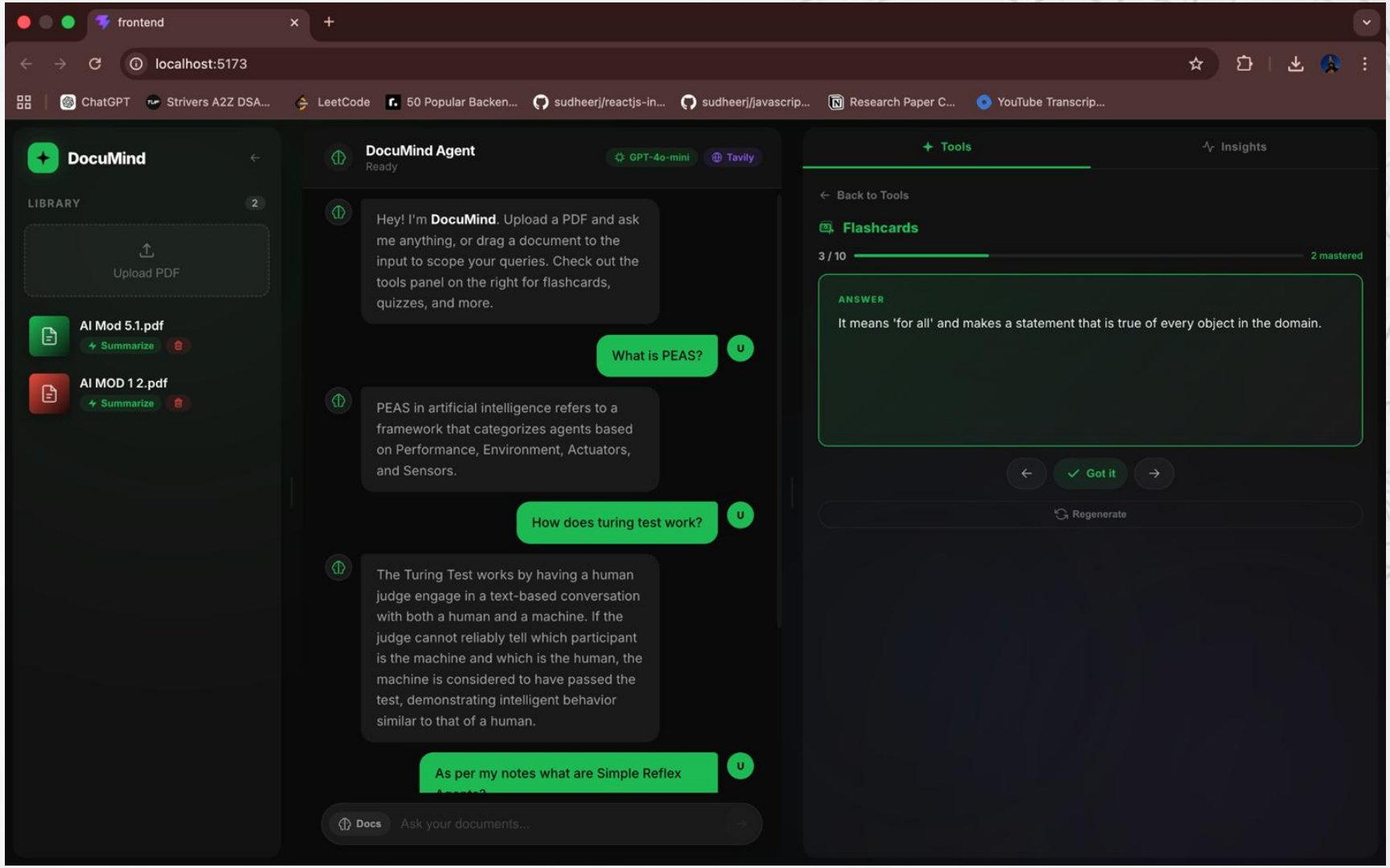
Key Concepts Feature

- Extracts important concepts from documents
- Uses semantic retrieval + LLM processing



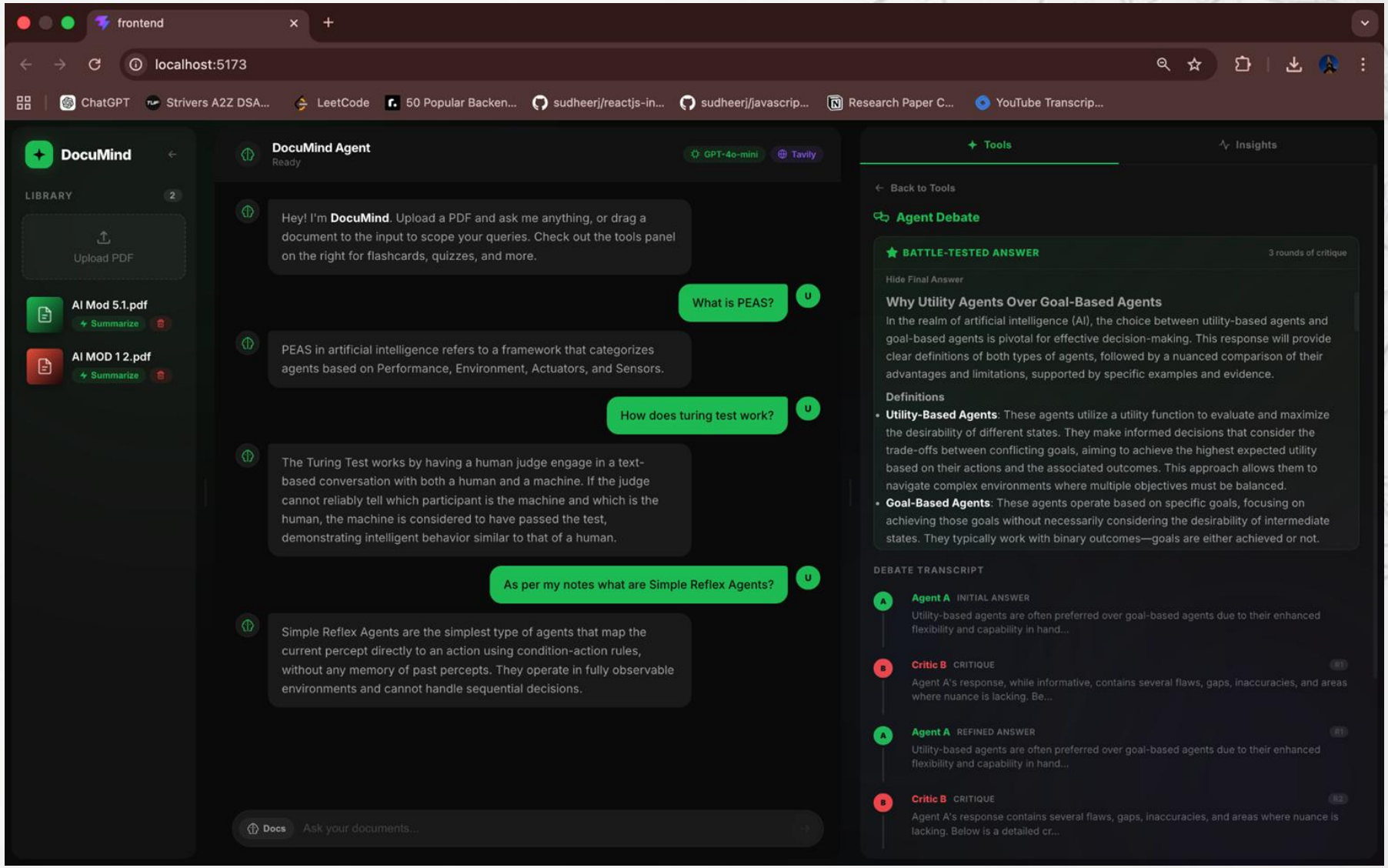
Flashcards Feature

- Generates Q&A pairs for revision
- Supports active recall learning



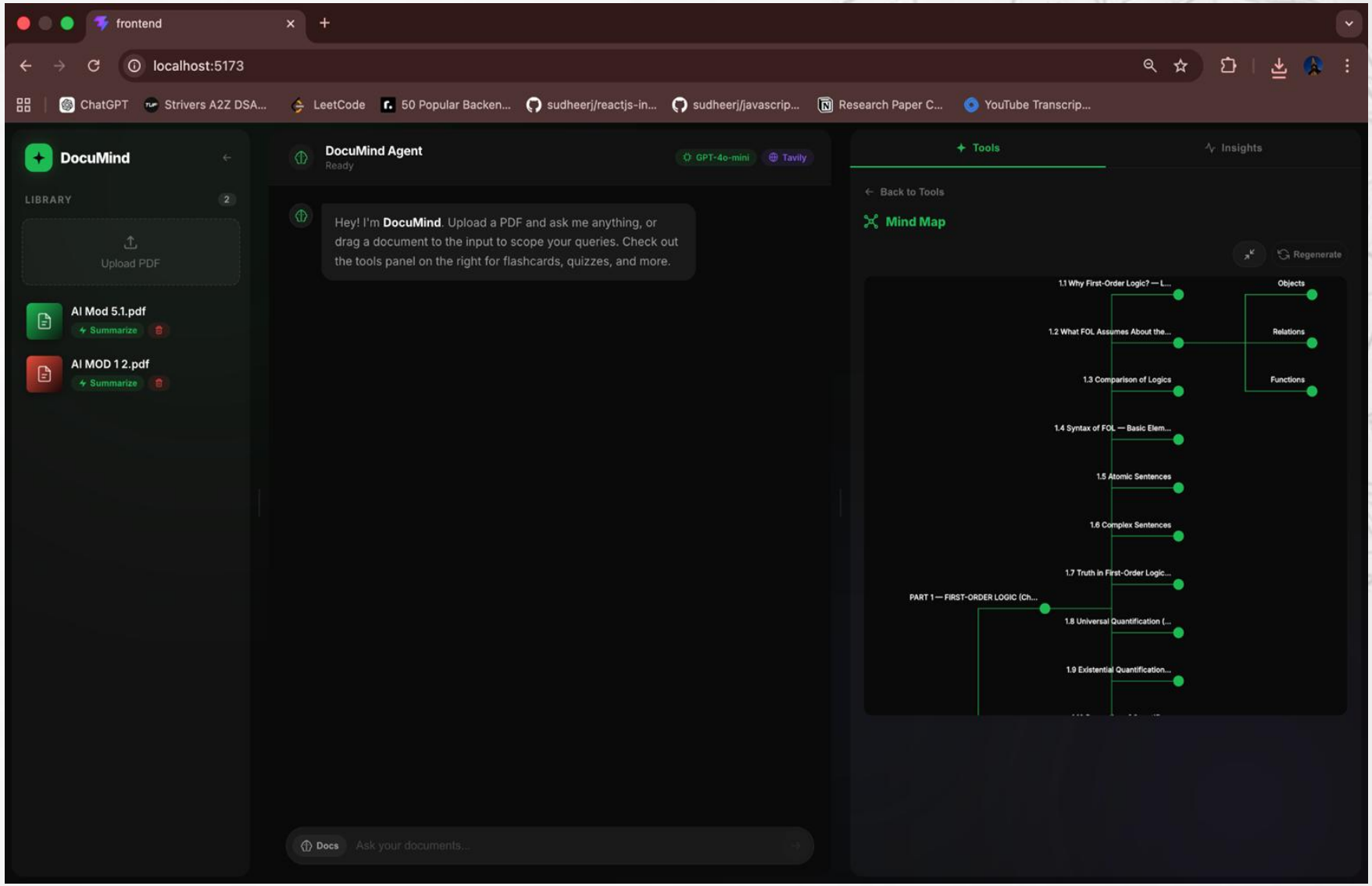
Agent Debate

- Simulates multi-agent discussion
- Enhances critical understanding of concepts



Mind Map

- Visualizes concept relationships
- Helps in structured learning and navigation



Classical vs Modern Agent

Feature	Classical Agent	DocuMind (Modern LLM Agent)
Decision Logic	Predefined rules (if-else, condition-action)	LLM-driven reasoning based on prompt understanding
Planning Strategy	Static, fixed execution path	Dynamic planning using **ReAct (Reason + Act)**
Memory	No memory or very limited state	Short-term contextual memory (chat history)
Execution Style	Single-step response	Multi-step reasoning with iterative refinement
Tool Usage	Hardcoded tool invocation	Autonomous tool selection via descriptions
Knowledge Source	Fixed knowledge base	Hybrid: **FAISS (documents) + Web search**
Adaptability	Cannot handle unseen queries well	Generalizes to new queries using LLM reasoning
Transparency	No visibility into decision process	Observable **Thought → Action → Observation logs**
Scalability	Difficult to extend (needs new rules)	Easily extensible by adding new tools